

Sektörde Yazılım Test Mühendisliği: Yapay Zeka Destekli API Test Otomasyonu Projesi



Yazılım Test Mühendisliği

Yazılım Test Mühendisliği, geliştirilen yazılımların gereksinimlere uygun çalıştığını doğrulamak ve olası hataları tespit etmek amacıyla gerçekleştirilen sistematik kalite güvence faaliyetidir.

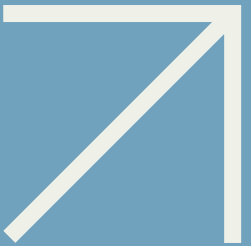
Yazılım hatalarını erken aşamada bulmak

Yazılım kalitesini artırmak

Sistem güvenilirliğini sağlamak

Müşteri memnuniyetini yükseltmek

Bakım ve düzeltme maliyetlerini azaltmak



Hata Kavramları ve Süreç Zinciri

Test faaliyetleri aşağıda gösterilen süreç zincirini mümkün olduğunca erken aşamada yakalamayı amaçlar.

Error (İnsan Hatası)

Geliştiricinin yaptığı yanlışlık



Fault / Defect (Yazılım Kusuru)

İnsan hatasının kod veya tasarıma yansımaları



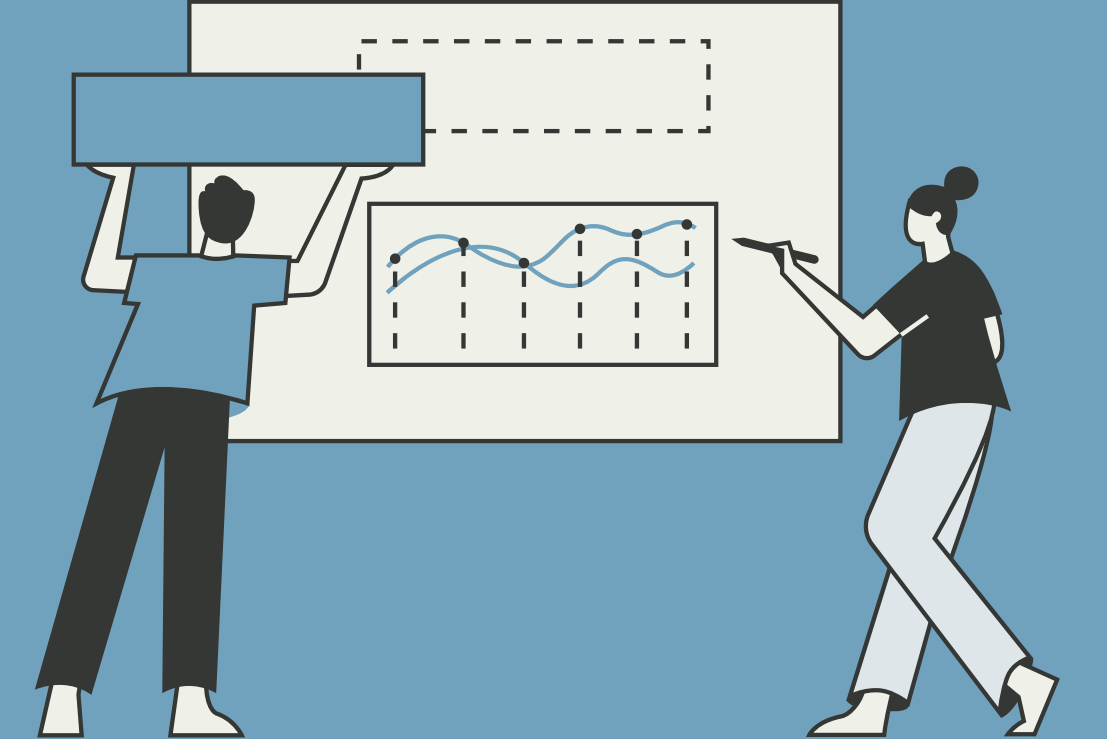
Failure (Başarısızlık)

Kusurun çalışma zamanında sistem davranışını bozması

Yazılım Test Seviyeleri

Amaç

Her seviyede farklı türde hataları tespit ederek yazılımın güvenilirliğini artırmaktır.



1

Birim Testi (Unit Test)

En küçük yazılım bileşenlerinin test edilmesi

2

Entegrasyon Testi

Modüller arası iletişimin doğrulanması

3

Sistem Testi

Tüm sistemin gereksinimlere göre test edilmesi

4

Kabul Testi (Acceptance Test)

Son kullanıcı tarafından gerçekleştirilen doğrulama süreci

Manuel ve Otomasyon Testleri

Manuel Test

Test senaryolarının insan tarafından yürütülmesi

Kullanıcı deneyiminin değerlendirilmesi

Esnek ve keşif odaklı yaklaşım

Otomasyon Testi

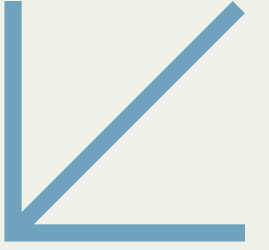
Testlerin araçlar yardımıyla otomatik çalıştırılması

Hızlı ve tekrarlanabilir sonuçlar

Özellikle regresyon testlerinde yaygın kullanım

Otomasyon testleri özellikle sürekli entegrasyon ve sürekli teslimat süreçlerinde büyük avantaj sağlar.

Yaygın Test Araçları



Selenium

WEB test otomasyon aracı.



Appium

Mobil test otomasyon aracı.



Postman

API test aracı.



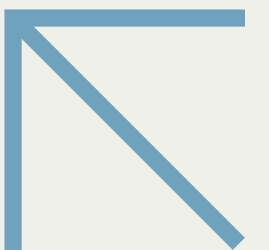
RestAssured

API otomasyon kütüphanesi.



JMeter

Performans test aracı.



Yapay Zeka Destekli API Test Otomasyonu Projesi

Proje Amacı

- Bankacılık API'lerinin otomatik olarak test edilmesi
- API davranışlarının doğrulanması
- Regresyon testlerinin otomatikleştirilmesi
- Performans ve veri doğruluğunun kontrol edilmesi

Proje Kapsamı

- Mock Bank API kullanılmıştır
- REST servisleri test edilmiştir
- GET ve POST endpointleri doğrulanmıştır
- Yapay zeka destekli geliştirme yaklaşımı uygulanmıştır

Bağlantı Adresi

<https://api.mockbank.io/swagger-ui.html>

Gerçek database'e bağlı değildir.

Test ve demo amaçlı geliştirilmiştir.

GET ve POST endpoint'leri içerir.

Bazı POST işlemleri kalıcı kayıt oluşturmaz

Response'lar statik veya sınırlı veri döndürebilir

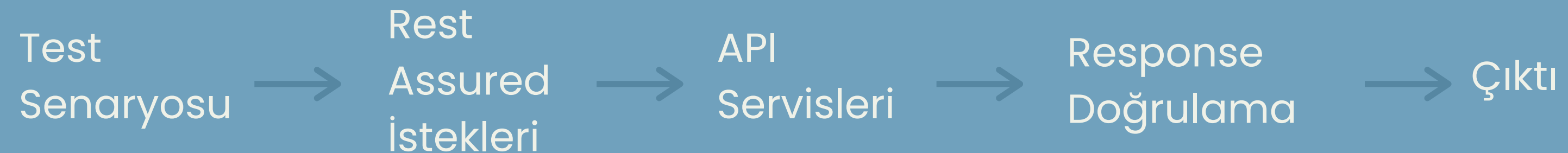


Mock Bank API

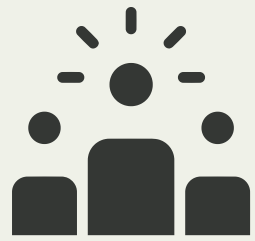
Neden bu API kullanıldı?

- Bankacılık sistemi simülasyonu yapmak
- API test otomasyon pratiği geliştirmek
- Gerçek sistem riskine girmeden test yapmak

Kullanılan Teknolojiler ve Mimari



Gerçekleştirilen API Testleri



Customer Servisi

- POST /customers
- GET /customers
- GET /customers/{id}
(Negative Test)



Transaction Servisi

- GET /transactions



Account Servisi

- GET /accounts

Testlerde Kontrol Edilenler:

- Status Code Doğrulama
- Response Body Doğrulama
- Response Time Kontrolü
- Negatif Senaryo Kontrolü

Yapay Zeka Desteđi ve Sonular

Yapay Zeka Kullanım Alanları

Test senaryolarının oluřturulması

Framework tasarımının geliştirilmesi

Kod geliştirme desteđi

Hata ayıklama süreçleri

Test kapsamının genişletilmesi

Proje ıktıları

API test otomasyonu geliştirildi

Rest Assured tabanlı framework oluřturuldu

GET ve POST testleri başarıyla alıřtırıldı

Veri dođrulama ve performans kontrolleri
gerekleřtirildi

Yazılım test mühendisliđi prensipleri
uygulandı

Teşekkür Ederim.

Beni dinlediğiniz için teşekkür ederim.

Bursa Teknik Üniversitesi, Bilgisayar Mühendisliği

Esra İLBOĞA 21360859063

